

Subarray XOR Sum Computations

Authored by: Arpit Raj, LNMIIT Jaipur | Date: October 4, 2026

© 2026 Arpit Raj. All rights reserved.

1. Problem Formulation & Prefix XORs

We need to compute the following summation over an array $a_1, a_2, a_3 \dots a_n$:

Formula: $\sum_{i=1}^n \sum_{j=1}^n f(i, j) \times (j - i + 1)$

Where: $f(i, j) = a_i \oplus a_{i+1} \oplus \dots \oplus a_j$

Modulo: 998244353

To compute this efficiently, we use the property of Prefix XORs. Let $\text{pref}[i]$ be the XOR of elements from index 1 to i :

- $\text{pref}[i] = a[0] \oplus a[1] \oplus a[2] \oplus \dots \oplus a[i]$ (The XOR of $[1 \dots i]$)
- $f(i, j) = \text{pref}[j] \oplus \text{pref}[i-1]$

Base cases for the prefix array:

- $\text{pref}[0] = 0$
- $\text{pref}[1] = a[0]$
- $\text{pref}[2] = a[0] \oplus a[1]$

2. Bitwise Contribution Analysis

For a particular bit b among $\text{pref}[0], \text{pref}[1] \dots \text{pref}[n]$:

Need to do diff (XOR logic)

Since $0 \oplus 1 = 1$ and $1 \oplus 0 = 1$, we only get a set bit when the corresponding bits in the two prefix XORs are different.

- **ones** = no. of prefix XORs whose bit b is 1
- **zeros** = no. of prefix XORs whose bit b is 0

Every 1 can pair with every 0. Therefore, the number of ways is $(\text{ones} \times \text{zeros})$.

4. Algorithm Steps & Variables

- **Step 1:** Find no. of bits needed.
- **Step 2:** Maintain variables for counts and sums:

Variable	Meaning	Initial State
cnt0 , sum0	Previous prefix XORs where bit is 0, and the sum of their indices	cnt0 = 1 , sum0 = 0 (b/c pref[0]=0)
cnt1 , sum1	Previous prefix XORs where bit is 1, and the sum of their indices	cnt1 = 0 , sum1 = 0

5. Source Code

```
#include<bits/stdc++.h>
using namespace std;

int main(){

    int n;
    cin>>n;

    vector<int> arr(n);
    for(int i=0;i<n;i++){
        cin>>arr[i];
    }

    vector<int> pref(n+1,0);
    for(int i=0;i<n;i++){
        pref[i+1]=pref[i]^arr[i];
    }

    int maxu=*max_element(arr.begin(),arr.end());
    int bits=1;

    while((1LL<<bits)<=maxu){
        bits++;
    }
    long long ans=0;
    const long long MOD=998244353;
```

5. Source Code (Continued)

```
for(int bit=0;bit<bits;bit++){

    long long cnt0=1;
    long long sum0=0;
    long long cnt1=0;
    long long sum1=0;

    for(int j=1;j<=n;j++){
        int curr=(pref[j]>>bit)&1;
        if(curr==0){

            long long ways=(cnt1*j)-sum1;
            ways%=MOD;
            ans=(ans+ways*(1LL<<bit))%MOD;
            cnt0++;
            sum0+=j;
        }

        else{
            long long ways=(cnt0*j)-sum0;
            ways%=MOD;
            ans=(ans+ways*(1LL<<bit))%MOD;
            cnt1++;
            sum1+=j;
        }
    }

    cout<<ans<<"\n";
    return 0;
}
```